DuoFlow: A Framework for Language-Guided Bimanual Manipulation

XJTU IAIR & WisdomStride

zhengyj@stu.xjtu.edu.cn

Abstract

Language-guided bimanual manipulation requires a policy to connect visual context, robot configuration, and a shared task goal with coordinated actions over time. We present DuoFlow, a framework that integrates multi-view observations, language conditioning, joint action-sequence generation, and receding-horizon execution through a fixed deployment interface. The interface exposes three RGB views and a 14-dimensional robot state, generates 50-step joint action sequences, and executes 30 steps before refreshing the observations. We study the framework using Clean demonstrations, independent scene confirmation, and broad task coverage. DuoFlow achieves 259 successes over 300 independent confirmation scenes, corresponding to an 86.33% success rate. In a complete 50-task local evaluation comprising 10,000 episodes, it achieves 76.41% mean success, with 75.64% in Local Clean and 77.18% in Local Randomized. We describe the deployed framework and report complete task-level success rates.

1 Introduction

Bimanual manipulation is a demanding setting for learned robotic control because success depends on the evolution of two arms within a shared scene. One arm may stabilize a container while the other places an object inside it; a handover requires the receiving arm to be positioned before the supporting grasp is released. Even when the final goal is simple to express in language, its execution involves a sequence of geometric and temporal decisions. Camera observations reveal only part of the relevant state, and an action that was appropriate when predicted can become unsuitable after an object moves or contact changes. The learning system and its execution interface must therefore be considered together.

Recent work has made temporally extended action prediction a practical way to connect visual observations with robot control. Action chunking predicts several future commands in one policy query, while generative policies represent distributions over action sequences. Large robot policies additionally use language and heterogeneous visual inputs to describe tasks beyond a single demonstration setting. These developments motivate a concrete systems question: how should a language-guided bimanual policy be deployed and evaluated so that its measured behavior can be interpreted across tasks and scene conditions?

We develop DuoFlow around an explicit perception-to-action contract. A policy query consumes three public RGB views, joint-state feedback, and a task instruction. The policy produces a joint sequence for both arms. A bounded prefix of that sequence is executed before a new observation is acquired and the prediction is refreshed. A separate evaluator manages scene construction, task termination, and success recording. This division makes the operational behavior of the system explicit: the policy determines robot actions from its allowed observations, while the evaluator determines which episodes are valid and how success is counted.

We evaluate DuoFlow through independent scene confirmation and a complete 50-task local evaluation. The development demonstrations cover three tasks, with 45 Clean trajectories per task. Independent confirmation contains 300 scenes, while the 50-task evaluation contains 100 episodes per task and visual condition, totaling 10,000 episodes. The deployed input-output interface and inference schedule remain fixed throughout evaluation.

We describe the DuoFlow observation interface, joint action representation, and execution schedule, and report complete task-level success rates on independent confirmation scenes and the 50-task evaluation.

2 Related Work

2.1 Bimanual learning and action sequences

Learning to manipulate with two arms requires a representation that can express complementary and synchronized behavior. ALOHA demonstrates the utility of low-cost bimanual teleoperation and Action Chunking with Transformers for fine-grained manipulation [8]. Its action-chunk representation is relevant to our deployment setting because a sin-

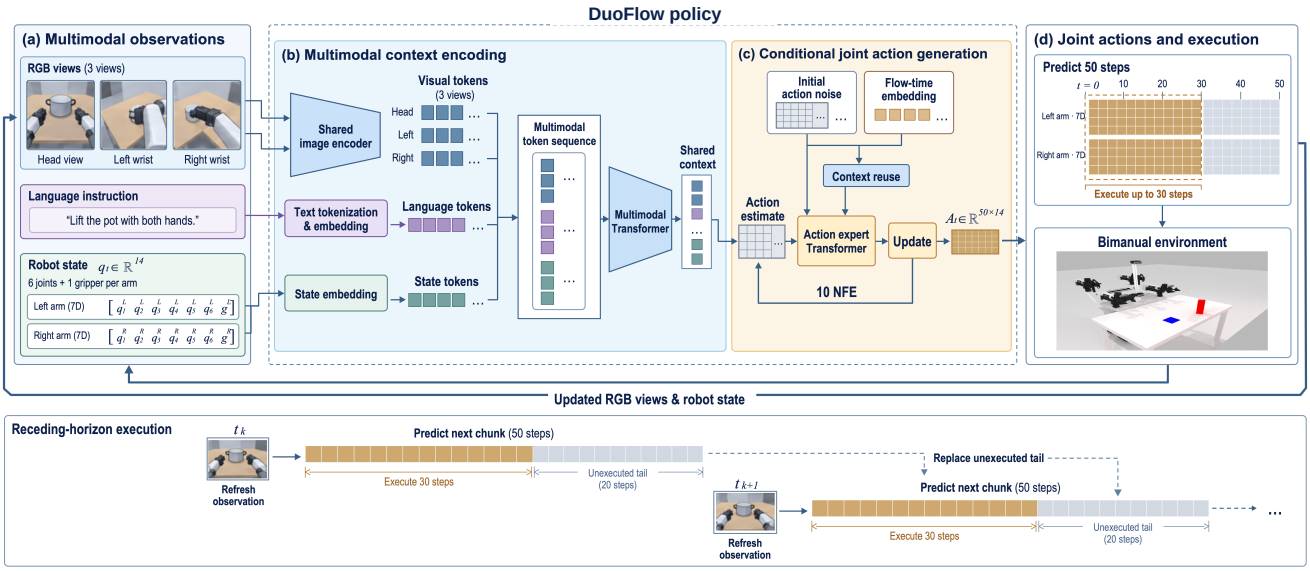


Figure 1: The deployed DuoFlow loop. Three RGB views, the instruction, and robot state condition a joint 50-step action sequence. Up to 30 actions are executed before refreshing observations. Scene admission and success checking belong to the evaluator; their internal signals are not policy inputs.

gle query specifies a temporally extended control segment. Jointly representing commands for the two arms also makes their intended timing explicit at the interface.

Diffusion Policy models visuomotor behavior as a conditional distribution over actions and combines sequence prediction with receding-horizon execution [3]. The broader lesson is that action generation and execution scheduling are distinct design choices. A model can predict a longer sequence than the portion executed before the next observation. DuoFlow uses this distinction operationally, with a prediction horizon of 50 and an execution horizon of 30.

2.2 Language-conditioned robot policies

Vision-language-action models connect robot control to representations that encode objects, scenes, and natural-language goals. RT-2 investigates transferring web-scale visual and linguistic knowledge into robotic action prediction [9]. OpenVLA presents an open vision-language-action model with adaptation to robot embodiments [5]. Octo studies generalist robot policies trained on heterogeneous robot data and conditioned through flexible task specifications [4]. These systems differ in architecture, action representation, data, and adaptation procedure, but they share the need for a clear contract between policy inputs and executable robot commands.

Flow-based robot policies further emphasize continuous action-sequence generation. The pi-zero work combines a vision-language backbone with flow-matching action generation for general robot control [1]. Our operational description is compatible with evaluating a generative action policy through a fixed number of model function evaluations.

The role of data is equally important. BridgeData V2 illustrates the value of diverse robotic manipulation data for generalization [6]. Broader datasets can differ substantially from a small adaptation corpus in task coverage, embodiment, and collection conditions.

2.3 Benchmarks and evaluation interfaces

RoboTwin 2.0 provides a large bimanual benchmark with scalable data collection and domain randomization [2]. Its clean and randomized scene conditions offer complementary views of policy behavior. These dimensions make the benchmark useful, but they also require precise reporting: an average is meaningful only when task coverage and evaluation conditions are specified together.

Evaluation interfaces such as XPolicyLab aim to standardize how a policy is connected to the environment [7]. We evaluate DuoFlow across the task set through a fixed observation and action interface.

3 DuoFlow Framework

3.1 Problem formulation and observation contract

We consider episodic, language-guided manipulation with two arms. Let g denote the natural-language instruction and let o_t denote the public observation at the time of a policy

query. The deployed observation consists of three RGB images and the current robot state:

$$o_t = (I_t^{(1)}, I_t^{(2)}, I_t^{(3)}, q_t), \qquad q_t \in \mathbb{R}^{14}. \tag{1}$$

The state contains six joint coordinates and one gripper coordinate for each arm. The images provide complementary views of the workspace. The instruction is fixed for an episode, while the images and robot state are refreshed during execution. These inputs supply visual context, a task description, and feedback about the current robot configuration. They do not provide privileged object poses or an expert's intended future actions.

We write the policy interface as a conditional generator $\pi_\theta(A_t \mid o_t, g)$, where θ denotes the frozen evaluated parameters and A_t is a sequence of joint commands. The policy conditions each sequence on the latest visual observations, robot configuration, and instruction.

The observation contract is also an information boundary. Simulator object transforms, scene-admission outcomes, expert trajectories, and success flags are not additional inputs to the learned policy. The evaluator may use privileged simulation information to construct and assess an episode, but the policy receives the specified public observations.

3.2 Joint action representation

The policy produces a sequence containing commands for both arms:

$$\begin{aligned} A_t &= (a_{t|t}, \ldots, a_{t+H-1|t}), \\ a_{t+j|t} &= [a^L_{t+j|t}; a^R_{t+j|t}], \quad H = 50. \end{aligned} \tag{2}$$

The subscript before the vertical bar identifies the intended execution step; the subscript after it identifies the observation used for prediction. The left and right components follow the robot's configured joint-and-gripper action convention. The deployment adapter preserves the expected ordering and converts the predicted output into the action dictionary consumed by the simulator. A valid command therefore refers to both arms at the same sequence index.

This joint representation expresses coordinated motion at the output interface. For a handover, it can represent approach, support, and release in a common sequence; for a two-arm lift, it can represent simultaneous motion.

The evaluated generator uses 10 model function evaluations per action prediction. This number is fixed across task conditions.

3.3 Receding-horizon execution

Only a prefix of the predicted sequence is applied before the policy is queried again. Let $E = 30$ be the execution horizon. At query index k, the evaluator applies up to E commands from A_{t_k}, subject to episode termination and the task's step limit. If e_k commands are actually executed, the next query occurs at

$$t_{k+1} = t_k + e_k, \qquad 0 < e_k \le E \le H. \tag{3}$$

The new prediction is conditioned on the updated observation. The unexecuted tail of the previous prediction is replaced by the next sequence. This provides feedback at a coarser rate than individual simulator actions while allowing the generator to express behavior beyond the immediately executed prefix. A longer prediction horizon and a shorter execution horizon are therefore compatible choices rather than conflicting specifications.

The execution schedule involves a tradeoff. Applying more actions per query can reduce the frequency of expensive policy calls, but it also increases the interval before visual feedback changes the predicted sequence. Querying more frequently can respond sooner to scene changes while incurring additional inference. The execution schedule is fixed across all evaluated tasks and visual conditions.

At the beginning of each episode, policy-side episode state is reset according to the adapter contract. During an episode, the deployment loop acquires observations, generates a joint sequence, applies the prescribed prefix, and obtains fresh observations. Success terminates the episode according to the native task rule.

3.4 Deployment and evaluator responsibilities

The deployment organization separates three responsibilities. The policy service performs model inference. The adapter maps the benchmark's observation and action interfaces to that service. The evaluator owns environment construction, instruction selection, scene admission, task-specific stopping conditions, and outcome recording.

Scene admission is an environment-side procedure executed before a policy trial. A scene rejected by the evaluator's validity check is not counted as a policy failure, because the evaluated policy has not yet acted. Once a scene is admitted, its success or failure contributes to the policy denominator. Privileged simulator signals used for admission and success checking are not policy inputs, and the deployment loop does not add corrective actions after a failed trial.

4 Experimental Design

4.1 Data and Checkpoint

Training-data Protocol and Initialization

DuoFlow follows the official RoboTwin 2.0 Clean-only training-data setting.

Table 1: DuoFlow success rates in the local 50-task evaluation. Each task has 100 episodes per condition.

Condition	Successful episodes	Evaluated episodes	Success rate
Local Clean	3,782	5,000	75.64%
Local Randomized	3,859	5,000	77.18%
Overall mean	7,641	10,000	76.41%

Table 2: DuoFlow success rates on 300 independent confirmation scenes. Each task has 50 scenes per condition.

Task	Local Clean	Local Randomized	Combined
Dump	46/50 (92%)	40/50 (80%)	86/100 (86%)
Handover	38/50 (76%)	42/50 (84%)	80/100 (80%)
Lift	50/50 (100%)	43/50 (86%)	93/100 (93%)
All scenes	134/150 (89.33%)	125/150 (83.33%)	259/300 (86.33%)

The benchmark provides 50 `demo_clean` demonstrations for each of the 50 Aloha-AgileX tasks, totaling 2,500 demonstrations.[1]

At this stage, we do not introduce any additional demonstrations, Randomized training trajectories, or evaluation rollouts.

4.2 Task Coverage

Independent confirmation covers three tasks under two visual conditions, with 50 scenes per task–condition pair, totaling 300 scenes.

The scene-seed range used for independent confirmation is separate from that of the preliminary screening stage.

The complete evaluation covers all 50 tasks under both Local Clean and Local Randomized conditions.

Each task–condition pair contains 100 valid policy episodes.

Consequently, the evaluation comprises 5,000 episodes per condition and 10,000 episodes in total.

Episodes used for independent confirmation are excluded from the 10,000-episode evaluation.

4.3 Success-rate metrics

For task u and condition c, let $s_{u,c,i}$ be one when valid policy episode i succeeds and zero otherwise. The task-level success rate is

$$\mathrm{SR}_{u,c} = \frac{1}{n_{u,c}} \sum_{i=1}^{n_{u,c}} s_{u,c,i}. \tag{4}$$

The condition success rate is the unweighted mean across the 50 tasks. The overall mean gives equal weight to the two conditions:

$$\begin{aligned} \mathrm{SR}_c &= \frac{1}{50} \sum_{u=1}^{50} \mathrm{SR}_{u,c}, \\ \mathrm{Mean} &= \frac{\mathrm{SR}_{\mathrm{Clean}} + \mathrm{SR}_{\mathrm{Randomized}}}{2}. \end{aligned} \tag{5}$$

With 100 episodes per cell, these means use 5,000 episodes per condition and 10,000 overall.

5 Results

5.1 50-task aggregate

DuoFlow achieves 7,641 successes over 10,000 episodes in the complete 50-task local evaluation, giving a mean success rate of 76.41%. It completes 3,782 of 5,000 Local Clean episodes (75.64%) and 3,859 of 5,000 Local Randomized episodes (77.18%). Table 1 summarizes these results. The overall mean weights the two conditions equally.

5.2 Task-level success rates

The task-level results cover the full set of 50 manipulation tasks. DuoFlow attains a mean success rate of at least 90% on 15 tasks and at least 80% on 25 tasks, with each task evaluated over 200 episodes. The complete results are visualized in Figure 2 and listed by condition in the supplementary table.

DuoFlow completes all 200 episodes on bell clicking, roller grasping, and horizontal bottle shaking. It records 188/200 successes (94%) on pot lifting, 174/200 (87%) on bin emptying, 165/200 (82.5%) on microwave opening, and 152/200 (76%) on block handover.

[1] https://robotwin-platform.github.io/leaderboard, accessed October 25, 2026.

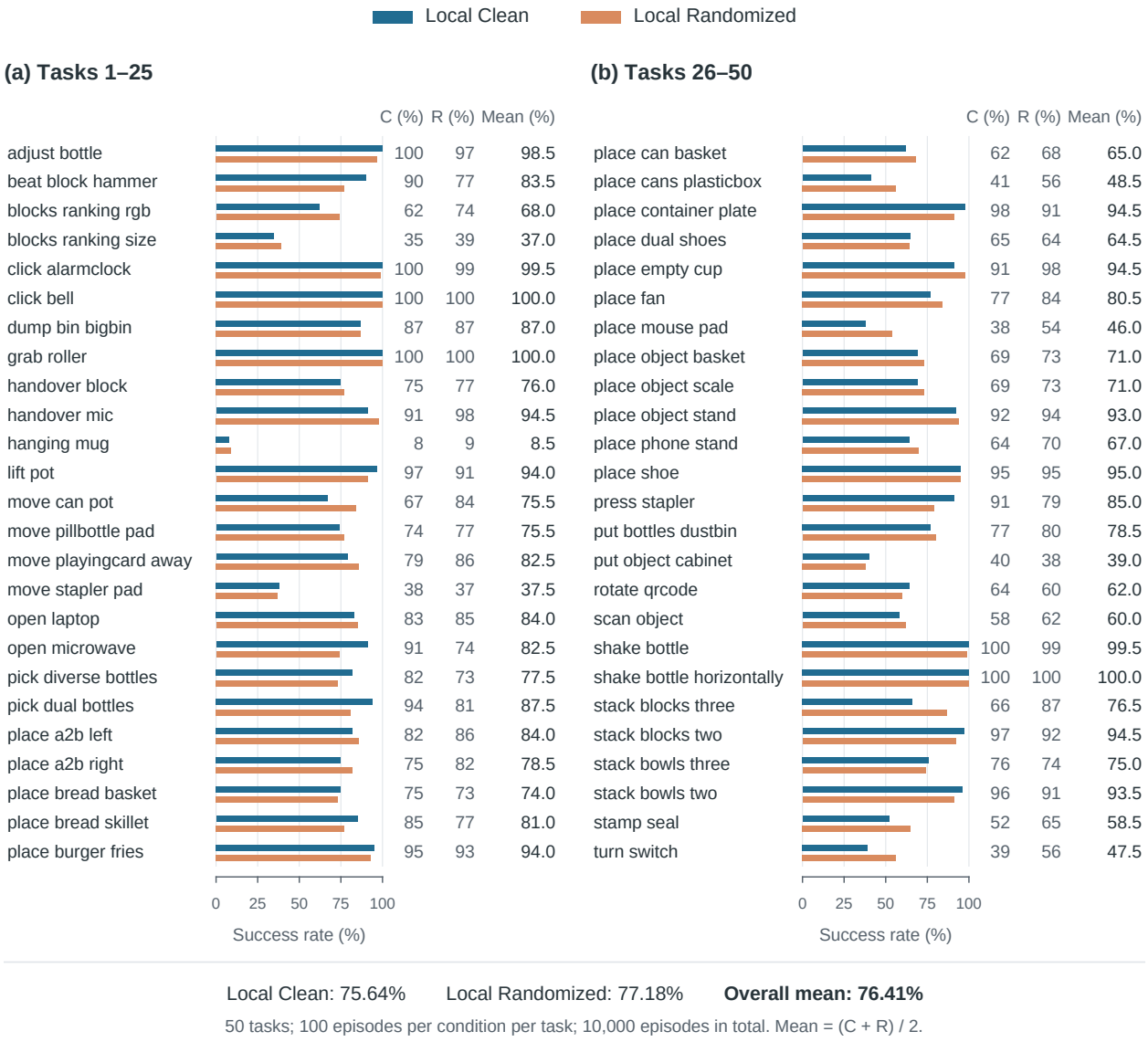


Figure 2: DuoFlow success rates across all 50 tasks. Paired bars show Local Clean and Local Randomized results; numerical columns report both conditions and their equally weighted mean. Each task has 100 episodes per condition. Tasks are listed alphabetically.

5.3 Independent scene confirmation

Independent confirmation yields 259 successful episodes over 300 scenes, corresponding to an 86.33% success rate. The checkpoint completes 86/100 scenes in dumping, 80/100 in handover, and 93/100 in lifting. The two condition totals are 134/150 (89.33%) in Local Clean and 125/150 (83.33%) in Local Randomized.

Table 2 provides the complete breakdown by task and condition. These confirmation scenes are separate from the 10,000-episode evaluation.

6 Conclusion

We present DuoFlow, a language-guided bimanual manipulation framework that integrates multi-view observations, joint action-sequence generation, and fixed receding-horizon execution. DuoFlow achieves 259/300 successes (86.33%) in independent scene confirmation and 76.41% mean success across the complete 50-task local evaluation, with 75.64% in Local Clean and 77.18% in Local Randomized. The task-level results cover all 50 evaluated manipulation tasks.

References

[1] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems XXI*, Los Angeles, CA, USA, 2025.

[2] Tianxing Chen, Zanxin Chen, Baijun Chen, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 13673–13699. PMLR, 2026.

[3] Cheng Chi, Zhenjia Xu, Siyuan Feng, et al. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10–11):1684–1704, 2025.

[4] Dibya Ghosh, Homer Rich Walke, Karl Pertsch, et al. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems XX*, Delft, Netherlands, 2024.

[5] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*, pages 2679–2713. PMLR, 2025.

[6] Homer Rich Walke, Kevin Black, Tony Z. Zhao, et al. BridgeData V2: A dataset for robot learning at scale. In *Proceedings of The 7th Conference on Robot Learning*, pages 1723–1736. PMLR, 2023.

[7] XPolicyLab. XPolicyLab: A unified standard and open ecosystem for robot policy evaluation and deployment. Software repository, 2026. Accessed 2026-10-19.

[8] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems XIX*, Daegu, Korea, 2023.

[9] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of The 7th Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

Supplementary Material: Complete Task Results

DuoFlow is evaluated for 100 episodes per task and local condition. Total successes are out of 200; mean success rates weight the two conditions equally.

Task	Clean SR	Randomized SR	Total successes	Mean SR
adjust_bottle	100%	97%	197/200	98.5%
beat_block_hammer	90%	77%	167/200	83.5%
blocks_ranking_rgb	62%	74%	136/200	68%
blocks_ranking_size	35%	39%	74/200	37%
click_alarmclock	100%	99%	199/200	99.5%
click_bell	100%	100%	200/200	100%
dump_bin_bigbin	87%	87%	174/200	87%
grab_roller	100%	100%	200/200	100%
handover_block	75%	77%	152/200	76%
handover_mic	91%	98%	189/200	94.5%
hanging_mug	8%	9%	17/200	8.5%
lift_pot	97%	91%	188/200	94%
move_can_pot	67%	84%	151/200	75.5%
move_pillbottle_pad	74%	77%	151/200	75.5%
move_playingcard_away	79%	86%	165/200	82.5%
move_stapler_pad	38%	37%	75/200	37.5%
open_laptop	83%	85%	168/200	84%
open_microwave	91%	74%	165/200	82.5%
pick_diverse_bottles	82%	73%	155/200	77.5%
pick_dual_bottles	94%	81%	175/200	87.5%
place_a2b_left	82%	86%	168/200	84%
place_a2b_right	75%	82%	157/200	78.5%
place_bread_basket	75%	73%	148/200	74%
place_bread_skillet	85%	77%	162/200	81%
place_burger_fries	95%	93%	188/200	94%
place_can_basket	62%	68%	130/200	65%
place_cans_plasticbox	41%	56%	97/200	48.5%
place_container_plate	98%	91%	189/200	94.5%
place_dual_shoes	65%	64%	129/200	64.5%
place_empty_cup	91%	98%	189/200	94.5%
place_fan	77%	84%	161/200	80.5%
place_mouse_pad	38%	54%	92/200	46%
place_object_basket	69%	73%	142/200	71%
place_object_scale	69%	73%	142/200	71%
place_object_stand	92%	94%	186/200	93%
place_phone_stand	64%	70%	134/200	67%
place_shoe	95%	95%	190/200	95%
press_stapler	91%	79%	170/200	85%
put_bottles_dustbin	77%	80%	157/200	78.5%
put_object_cabinet	40%	38%	78/200	39%
rotate_qrcode	64%	60%	124/200	62%
scan_object	58%	62%	120/200	60%
shake_bottle	100%	99%	199/200	99.5%
shake_bottle_horizontally	100%	100%	200/200	100%
stack_blocks_three	66%	87%	153/200	76.5%
stack_blocks_two	97%	92%	189/200	94.5%
stack_bowls_three	76%	74%	150/200	75%
stack_bowls_two	96%	91%	187/200	93.5%
stamp_seal	52%	65%	117/200	58.5%
turn_switch	39%	56%	95/200	47.5%